

Extraction and Representation of Sparsity Patterns for Efficient Data Transfer on Accelerators

Yang Su[†], Toshiyuki Ichiba[‡], Katsuhiro Yoda[‡], Yasuhiro Watanabe[‡],
Takahide Yoshikawa[‡] Tarek S. Abdelrahman[†]

[†]University of Toronto, Canada
yangg.su@mail.utoronto.ca,
tsa@ece.utoronto.ca

[‡]Fujitsu Limited, Japan
{t.ichiba, yoda.katsuhiro, yas.watanabe,
yoshikawa.takahide}@fujitsu.com

Abstract—Sparse computations are common in practical HPC, AI and graph-based applications. Such computations often exhibit scattered and fragmented data accesses, which negatively impact data transfer efficiency to/from accelerators. We propose, implement and evaluate an algorithm for *extracting or mining* sparsity patterns that exist in sparse matrices. The algorithm extracts multiple pattern types in a matrix, including blocks, bands, triangles or regular compositions of each. It does so without a priori knowledge of the presence of these patterns in the matrix. The patterns may contain, under user control, zero elements, or *imperfections*, to facilitate the extraction of larger patterns. Additionally, we introduce the *Compressed Sparse Pattern* (CSP), a novel compressed representation for sparse matrices that is based on these patterns. The use of CSP combined with extensions to Address Generation Units (AGUs) of accelerators regularize data accesses and improve data transfer efficiency. Evaluation of the pattern mining algorithm and CSP using 26 real-world sparse matrices is conducted on an Ubuntu system with an 8 core Intel CPU (3.6 GHz i7-9700K) and 32 GB of memory. The evaluation shows that patterns of different sizes and shapes are common, representing $\sim 82\%$ of the non-zero elements in these matrices. The patterns can be efficiently extracted in time, with an average of 4.6 seconds. The evaluation also shows that the mining of composite patterns contributes $\sim 8\%$ to the number of non-zeros in patterns and that imperfections increase pattern sizes with a minimal impact of only $\sim 7\%$ zero elements in patterns. Finally, using CSP leads to up to 90% reduction in data transfer overhead, compared to CSR and CSC, both common compressed sparse matrix representations. These results validate our approach of extracting and representing patterns to improve data transfer efficiency.

Index Terms—sparsity patterns, sparse matrix representations, data transfer efficiency, heterogeneous systems, accelerators

I. INTRODUCTION

Sparse matrices are common in HPC workloads, such as molecular dynamics [1], circuit analysis [2], quantum chemistry [3] and finite elements [4]. They are also increasingly becoming common in deep learning workloads with the advent of sparse large language models [5]–[7]. Increasingly, these workloads are processed by specialized *accelerators* that can deliver an order-of-magnitude performance improvement over their *host* multicores.

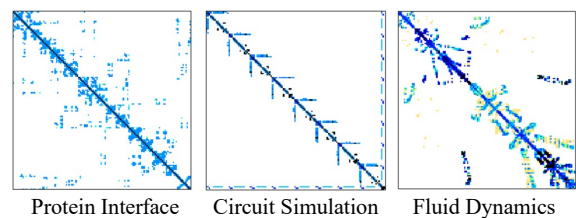
Sparse matrices are typically represented in one of several compact representations, such as compressed sparse row (CSR) [8], ELL-PACK [9] or SELL-C- σ [10]. These representations store only (or mostly) the non-zero elements of a sparse matrix. However, they require the use of metadata in the form of *index* or *overhead* arrays to determine the location

of the non-zero elements within the matrix. The use of such metadata often leads to indirect array accesses, which in turn lead to scattered and fragmented memory accesses.

Scattered and fragmented memory accesses challenge accelerators, which rely on high-speed data transfer between the host's system memory and the accelerator's local memory. In their presence, data is transferred either: (1) using multiple short data transfer operations, (2) using fewer such operations that include data not needed by the accelerator, or (3) by gathering the scattered data into a contiguous buffer and using a single operation. In all cases, overhead is introduced, which degrades the efficiency of data transfer.

Many sparse matrices have regularity in the distribution of their non-zero elements. Such regularity is referred to as a *sparsity pattern*. Sparse matrices that exhibit such patterns include block, diagonal, block-diagonal, and tri-band matrices [11]. More efficient *pattern-aware* compact sparse representations exist for such matrices, such as Block Compressed Sparse Row [12] and compressed diagonal storage [11]. These pattern-aware representations reduce or even eliminate the use of overhead arrays and lead to regular memory accesses.

However, most existing pattern-aware representations [11]–[16] are limited to matrices with a single pattern category, e.g., blocks or diagonals. Further, they require a priori knowledge that such type of pattern exists in a matrix. However, many sparse matrices exhibit patterns that do not fall into a single category. Figure 1 shows three example sparse matrices from the SuiteSparse Matrix Collection [17]. It shows patterns to their non-zero elements, but ones that cannot be represented by existing pattern-aware representations.



source: <https://sparse.tamu.edu/>
Fig. 1: Patterns in example sparse matrices

We propose, implement, and evaluate a *pattern mining* algorithm that extracts patterns from sparse matrices and

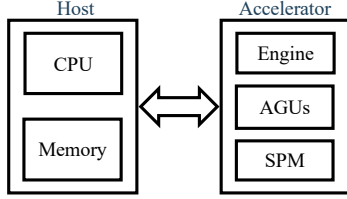


Fig. 2: Accelerator model

characterizes these patterns with a set of parameters. The patterns are not limited to a single pattern category and may be a mix of blocks, triangles, bands, or a composite of each. The algorithm extracts these patterns without a priori knowledge of their presence in a matrix. Further, we propose a novel compressed sparse matrix representation called *Compressed Sparse Pattern* (CSP) that stores pattern parameters as well as the non-zero elements of each pattern contiguously. We extend the Address Generation Units (AGUs) common in accelerators to utilize pattern parameters and CSP to regularize the otherwise scattered data accesses, leading to efficient data transfers to/from accelerators.

We evaluate the mining algorithm and using CSP with 26 real-world sparse matrices. The evaluation shows that: (1) sparsity patterns do exist in matrices in abundance, in different shapes and sizes, and their elements cover a significant portion of all non-zero elements, $\sim 82\%$, (2) patterns can be mined efficiently in time, taking on average 4.6 seconds and no more than ~ 1 minute, (3) the use of CSP results in significant improvements, by up to 90%, in data transfer efficiency compared to common representations such as Compressed Sparse Row (CSR) and Compressed Sparse Column (CSC).

Thus, the contributions of this work are:

- A novel pattern mining algorithm that efficiently extracts patterns in sparse matrices and represents them with parameters.
- A novel Compressed sparse pattern (CSP) representation that stores non-zero elements based on patterns.
- An extended AGU design that utilizes patterns parameters and CSP to regularize data accesses.
- An evaluation using real-world matrices that demonstrates the abundance of patterns and the utility of CSP in eliminating scattered accesses.

The remainder of this paper is organized as follows. Section II gives background material. Section III details the sparsity patterns we capture. Section IV presents the CSP representation. Sections V and VI outline how CSP is utilized and the necessary AGU extensions. Section VII gives our pattern mining algorithm. Section VIII presents our evaluation. Section IX reviews related work. Finally, Section X provides some concluding remarks and directions for future work.

II. BACKGROUND

A. Data transfer for accelerators

Accelerators have become a norm in HPC and machine learning (ML) workloads. Examples include NTX [18], NP-CGRA [19], HiPreP [20], DaVinci [21], EyerissV2 [22], AMD's Versal [23] to name a few (see [24] for a survey).

Accelerators vary in their designs, target applications, and performance. However, they share common architectural characteristics. First, they have *compute engines* capable of delivering high performance, especially for matrix multiplication computations. Second, they have *accelerator-local memory*, on the order of hundreds of Kbytes [19], [22] to 1 Mbytes [21], [25]–[27], that hold data for and results from the compute engine.

Third, these accelerators have hardware components, often referred to as *Address Generation Units* (AGUs) [18], [20], [23], [28] or *streaming units* [19], [22]. Each AGU generates a stream of local memory addresses to forward data from the local memory to the compute engine. This address stream is generated based on a set of *parameters*, typically the start address of the data in local memory, the size of the data, the stride of access, and the data count [29], [30].

Finally, the accelerators are attached to a CPU-based host with a high-speed data transfer mechanism, such as DMA, to copy data into and out of the accelerator's local memory. Our focus is on this data transfer. Thus, we abstract accelerators to only capture relevant components, as shown in Figure 2.

The abstracted accelerator consists of a compute engine, an accelerator-local memory referred to as the *scratchpad memory* (SPM), and a set of AGUs. The accelerator is connected to a host with a high-speed data transfer mechanism. The host's CPU programs the accelerator's engine with the desired computations. It then uses high-speed data transfer operations to copy needed data from its system memory to the SPM. The data is copied so it is *contiguous* in the SPM. This allows the CPU to program the AGUs to generate the addresses of the data in the SPM and to stream it to the engine. The CPU finally triggers the accelerator to start. Once done, results from the SPM are transferred to system memory.

B. Sparse Matrix Representations

Sparse matrix representations store only (or mostly) the non-zero elements of a matrix to reduce storage and eliminate operations with zero-operands. However, they introduce *overhead* or *index* arrays that map the non-zero values to their location in the sparse matrix.

For example, the Coordinate List representation (COO) [31] stores the non-zero elements unordered but contiguously in a *value* array and uses two arrays that store the row and column coordinates of each element. Similarly, the Compressed Sparse Row representation (CSR) [8] stores non-zero elements that are in the same row contiguously in a value array. Two overhead arrays store the row offsets and the column indices of the elements. Other representations [8]–[10] also introduce similar overhead arrays.

The use of overhead arrays results in indirect array accesses, which in turn result in data accesses that are scattered in memory. This is illustrated using the key loop of the ubiquitous sparse-matrix dense-vector (spMV) multiplication shown in Figure 3a, where the sparse matrix is stored in COO representation. Although the non-zero elements of the matrix A are stored contiguously in v , elements in x and y corresponding to these non-zero elements are located using the *row* and *col* arrays. Figure 3b shows an example sparse matrix and

how accesses to x and y become scattered, which affects data transfer efficiency as described below.

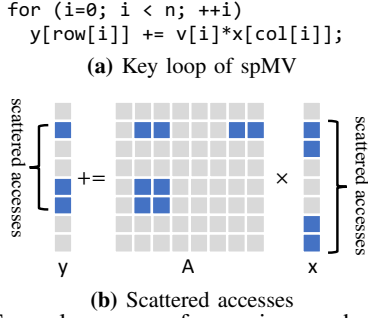


Fig. 3: Example accesses from using overhead arrays

The efficiency of data transfer depends on the number of high-speed data transfer operations the CPU must perform, and on the total size of the data transferred. When data is contiguous in system memory, an efficient single long transfer operation can be used. However, when data is scattered across memory, as is the case with a sparse computation, multiple short transfers are required, reducing data transfer efficiency.

III. SPARSITY PATTERNS

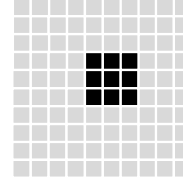
A sparsity pattern is a regularity in the distribution of non-zero elements in a sparse matrix. This regularity makes it possible to determine the locations of non-zero elements within a pattern. Example patterns include blocks, diagonals and bands.

An *affine* pattern is one for which the locations of its elements can be generated by an *affine loop nest*, i.e., a nest for which its loop bounds and its body contain only linear expressions of loop iterators [32]. Affine loop nests conveniently capture the sequence of row and column index pairs while exposing key parameters of affine patterns. These parameters lend themselves to a simple AGU design for reproducing the address sequences. Figure 4 shows an example block pattern and its corresponding affine nest. The sequence of `row` and `col` values generated by the loop's execution are the row and column coordinates of the non-zero elements.

A. Basic Affine Patterns

A basic affine pattern has its elements in a contiguous region in which each non-zero element can reach every other non-zero element by a sequence of movements to adjoining non-zero elements horizontally, vertically, or diagonally. Thus, within a basic pattern, no collection of zero elements divides the pattern into two or more contiguous regions.

A basic pattern may contain a limited number of zero elements that are allowed to avoid fragmenting patterns. These elements are treated as non-zeros. We refer to them as pattern *imperfections*. These imperfections can improve pattern sizes, but result in explicit storage of zeros and in unnecessary computations involving zeros. The *degree of imperfection* is the ratio of such zeros to the number of non-zero elements in a pattern, and it is kept low.

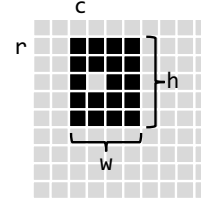


(a) A block pattern

```
int row, col;
for(int i=0; i<3; i++) {
  for(int j=0; j<3; j++) {
    row=3+i;
    col=4+j;
  }
}
```

(b) Corresponding loops

Fig. 4: A block pattern and its corresponding loop



(a) Pattern parameters

```
int row, col;
for(int i=0; i<h; i++) {
  for(int j=0; j<w; j++) {
    row=r+i;
    col=c+j;
  }
}
```

(b) Affine loops

Fig. 5: Block pattern parameters and affine loops

Basic patterns are characterized by parameters that describe their *position* and *shape*. Position parameters define the location of the non-zero element within a pattern that has the smallest sum of its row and column locations. Visually, this is the top left corner of the pattern. We refer to this element as the *start* of the pattern. Thus, the position parameters are the row index, r , and the column index, c , of this start element. Shape parameters describe the type, size, and orientation of a pattern. Depending on the type of the pattern, different numbers of shape parameters may be needed.

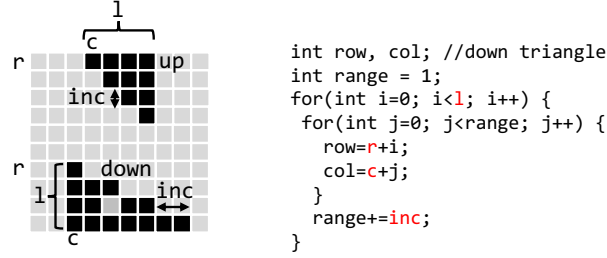
We extract *block*, *triangular* and *band* basic affine patterns.

1) *Block Patterns*: a block pattern has the shape of a horizontal line, a vertical line, or a rectangle that is aligned with the rows and columns of the matrix. Its shape is represented by its width (number of columns) w and its height (number of rows) h . Figure 5 illustrates these parameters and the affine nest that uses these parameters to generate element coordinates.

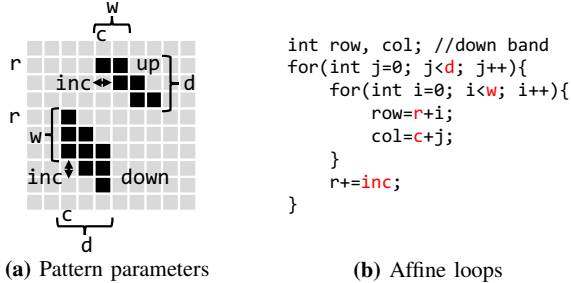
2) *Triangular Patterns*: these patterns often appear along diagonals of sparse matrices. Thus, we only extract right-angle triangles with their hypotenuse on a diagonal of the matrix. The shape of a triangular pattern is defined by its orientation, length (l), and increment (inc). Triangles have two possible orientations, *up* and *down*, as determined by how elements are adjacent to the start element of the pattern. If the adjacent elements are along the same row, the orientation is *up*. If the adjacent elements are along the same column, the orientation is *down*. The length of the pattern is the number of elements along its orientation. The increment is the change in position of elements against the orientation, i.e., columns for up and rows for down.

Figure 6 illustrates these parameters with two triangular patterns (up and down) and shows the affine loop that generates the coordinates of the elements for the *down* pattern. The loop nest is similar to the one for a block, with the exception that the index of the first element is increased by inc on successive iterations of the outer loop.

3) *Band Patterns*: are bands of non-zero elements in the form of parallelograms with one side parallel to the rows or



(a) Pattern parameters (b) Affine loops
Fig. 6: Triangular pattern parameters and affine loops



(a) Pattern parameters (b) Affine loops
Fig. 7: Band pattern parameters and affine loops

columns of the matrix. Bands represent diagonals of various locations, widths, and depths.

The shape of a band pattern is defined by its orientation, depth (d), width (w), and increment (inc). Bands have two possible orientations, *up* and *down*, determined by the direction of elements adjacent to the start element. Orientation is *up* if the adjacent elements are along the row; otherwise, it is down.

The width of the pattern is the number of elements along its orientation, while its depth is the number of elements orthogonal to the orientation. Simple diagonals have a width of 1. The change in the position of elements orthogonal to the orientation defines the increment of the band.

Figure 7 illustrates band pattern parameters with two patterns and shows the affine loops that generate the coordinates of the elements for down bands.

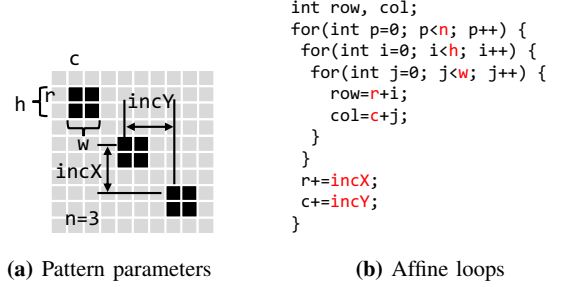
B. Composite Patterns

Composite patterns consist of n regular repetitions of basic patterns of the same type, shape, and size. A common example is diagonal blocks. We refer to the basic pattern as the *sub-pattern* of the composite pattern. The positions of these sub-patterns are incremented by $incX$ and $incY$. Figure 8 shows an example of a composite pattern, its parameters, and the affine loop nest that generates the locations of its elements.

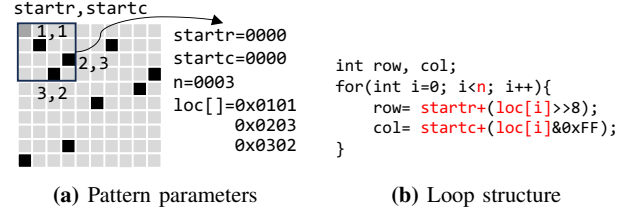
C. Non-Affine Patterns

In general, not all non-zero elements will belong to an affine pattern. Thus, we introduce *non-affine* patterns to capture non-zero elements that do not form basic or composite patterns.

Each non-affine pattern contains the non-zero elements in a fixed-sized square area in the sparse matrix. The start element of the square area is defined analogously to an affine pattern. The location of each non-zero element within the square area is stored as an offset from the start element. Thus, the parameters of a non-affine pattern are the row and column indices of its



(a) Pattern parameters (b) Affine loops
Fig. 8: Composite pattern parameters and affine loops



(a) Pattern parameters (b) Loop structure
Fig. 9: Non-affine pattern parameters and affine loop

start element, the number of non-zero elements, and an array that stores the offsets.

Figure 9 gives an example non-affine pattern. It shows a 4×4 square area with non-zero elements that do not form an affine pattern, the start element and the offsets of the non-zero elements in loc . The row and column indices of each offset are packed into a single integer.

IV. CSP REPRESENTATION

The Compressed Sparse Pattern (CSP) representation compresses a sparse matrix based on mined patterns to eliminate indirect array accesses. It stores the values of elements in a pattern contiguously, so that all pattern elements can be processed together, irrespective of their location in the matrix. This is in contrast to existing compressed representations, such as CSR or CSC, which store non-zero elements in rows or columns contiguously.

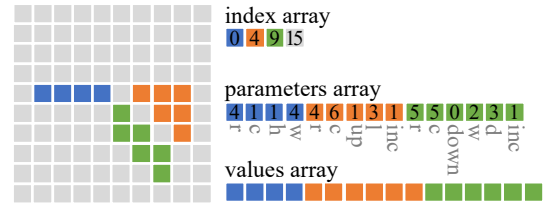


Fig. 10: An example matrix and its CSP representation

A CSP representation consists of three arrays. The first is the *parameters* array, and it stores the parameters of the patterns, one pattern after another. Since the number of parameters is different for each pattern, the *index* array stores the start locations of the parameters of each pattern in the parameters array. The type of pattern can be derived from the number of parameters. Finally, the *values* array stores the values of the non-zero elements of each pattern. Figure 10 shows an example sparse matrix and its CSP representation.

The key feature of CSP is that elements that belong to a pattern are stored contiguously in the values array, irrespective of the elements' locations in the matrix. This enables the iteration over elements of a pattern with one set of parameters, making it possible to regularize and directly determine the locations of elements in indirectly accessed arrays, eliminating the indirection, as is described in Section V.

V. UTILIZING CSP

The use of CSP is illustrated with the spMV code of Figure 3a, where COO requires indirect memory accesses and results in irregular memory accesses for x and y .

Figure 11 shows an example sparse matrix ① that is stored in the host memory in the CSP representation ②. There are two patterns: p_1 and p_2 . In order to utilize CSP with an accelerator, the spMV computation is structured as shown in the figure ③. Each pattern is processed in order by the host CPU. The steps of this processing are illustrated with p_2 .

The CPU extracts the parameters of p_2 (line 2). It uses these parameters to determine the section of A in the values array, and the sections of x and y corresponding to this values section in the spMV computation. These sections, which are each contiguous in the host memory, are sent to the accelerator's SPM as three contiguous buffers, v_{spm} , x_{spm} and y_{spm} , using 3 high speed data transfer operations ④ (line 3).

The CPU programs the accelerator's engine with the spMV computations, re-written to utilize the three SPM buffers ⑤ (line 4). It also uses the pattern parameters to program 3 AGUs, one for each of v_{spm} , x_{spm} and y_{spm} ⑥ (line 5).

The accelerator is started (line 6), and its AGUs generate the offsets from the start addresses of x_{spm} , y_{spm} and v_{spm} ⑦ in the SPM, streaming their data to the engine. Once the computation is done, the computed elements of y_{spm} are copied back to system memory ⑧ (line 8).

The key advantage of CSP is at the data transfer step ④ (line 3). Only the parameters of a pattern are needed to determine the segments of data in x , y corresponding to pattern elements in the values array. Combined with the AGUs, the need for indirect array accesses is eliminated. Memory accesses are regularized because the sections of values, x and y are contiguous.

However, the AGUs must generate the correct sequence of offsets for these elements in the SPM ⑦. The basic AGU design described in Section II can only generate a linear sequence of offsets that sweep over the elements of the buffers. However, with patterns, elements in some of the buffers may be reused ⑦ as determined by the type of the pattern. Thus, the basic AGU design must be extended to support patterns.

VI. EXTENDED AGU DESIGN

Figure 12 depicts the extended AGU design. Components highlighted in yellow stem from the basic AGU design of Section II, namely the base address, starting offset from this address (S), the stride (Str) and the count (C). It has a counter r , initialized to 0 and incremented by 1 every cycle C times. The basic AGU calculates $r \times Str + S$ to generate offsets, which are added to the base address of a buffer.

Components highlighted in red represent our extensions, which include several new inputs stored in registers, a set of

adders, multiplexers, comparators, and a local memory. The new inputs are: C_{comp} , the sub-pattern size; S_{comp} , the starting offset for composite pattern; ΔS and ΔS_{comp} , the change of starting offsets; T , the reset threshold; ΔT , the change of reset threshold; $isAffine$, a flag that indicates if the pattern is affine. These inputs are all derived from pattern parameters [33]. Some input registers, such as T and S_{comp} , are initialized with values but are updated dynamically. The offset S is also updated in the extended design.

The AGU extensions support different pattern types. For example, T is used to reset r to generate repeating offsets such as i_c ⑦ as the AGU generates addresses of elements on different rows or columns, depending on pattern type. The number of cycles between resets changes in triangles is inc , and ΔT specifies it. The offset can also change after each reset in bands (inc), hence summing S and ΔS is needed. ΔS is also used to increment offsets when Str is 0 to generate repeating offsets.

When generating offsets of a composite pattern, the offset changes between different sub-patterns ($incX$, $incY$), hence summing S_{comp} and ΔS_{comp} . Lastly, the loc vector of each non-affine pattern is processed by the host to isolate row and column indices, which are stored in the local memory of the AGU for index generation of the elements. The start point coordinates are the offsets in this case. r is used to index into the local memory. The flag $isAffine$ selects the local memory when the pattern is not affine.

For example, to generate offsets to y_{spm} for pattern p_2 , i.e., i_r ⑦, the inputs to the AGU are: $C = 6$; $Str = 0$; $T = 2$ and $\Delta T = 0$ as the block spans 3 columns and it does not change between different rows; S is 0 while ΔS is 1 since i_r increases by one after r is reset; S_{comp} , ΔS_{comp} and C_{comp} are not needed, and are set to 0; and $isAffine$ is *true*.

The overhead of the AGU extensions is reflected in Figure 12 as the "Extended" components. The exact impact of this overhead on area and power requires a specific accelerator design and technology, which is beyond the scope of this work. Nonetheless, we estimate this impact by calculating the number of NAND gates needed to implement the AGU. The basic AGU requires ~ 4400 gates. The number of gates needed for each extension is determined, e.g., ~ 300 gates for a comparator and ~ 800 gates for a 32-bit register with synchronous reset. The number of gates of the extended AGU is $\sim 3X-4X$ that of the basic AGU. The increase is well justified by obtained benefits.

VII. PATTERN MINING

Pattern mining is the process of finding a distribution of non-zero elements that can be collectively represented by a pattern, affine, composite or non-affine, and store them in a corresponding CSP representation. A high-level overview of this process is shown in Algorithm 1.

The algorithm takes a sparse matrix M in COO representation and produces the sparse matrix P in its CSP representation plus a boolean flag validating the conversion.

The matrix is converted into a container-based representation (line 1) that is more conducive to mining. Basic patterns are then iteratively mined (lines 2-5). Patterns smaller than a user-defined size are isolated (line 6-7) to build composite

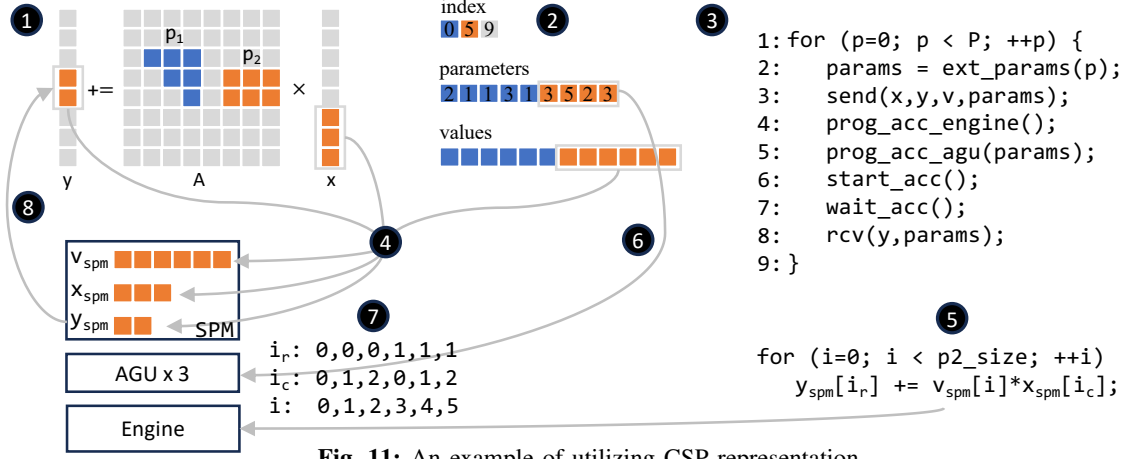


Fig. 11: An example of utilizing CSP representation

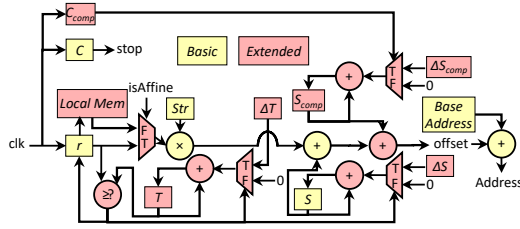


Fig. 12: Extended AGU design

Algorithm 1 Pattern Mining Algorithm

In: M

Out: P , valid

```

1:  $M_c \leftarrow \text{convert}(M)$ 
2: while  $M_c$  is not empty do
3:    $p \leftarrow \text{buildBasic}(M_c)$ 
4:    $M_c \leftarrow M_c - p$ 
5:    $P \leftarrow P \cup p$ 
6:  $\text{small}P \leftarrow \text{filterSmall}(P)$ 
7:  $P \leftarrow P - \text{small}P$ 
8:  $\text{comp}P, \text{unused}P \leftarrow \text{buildComposite}(\text{small}P)$ 
9:  $\text{verySmall}P \leftarrow \text{filterVSmall}(\text{unused}P)$ 
10:  $\text{small}P \leftarrow \text{unused}P - \text{verySmall}P$ 
11:  $\text{nonAffine}P \leftarrow \text{buildNonAffine}(\text{verySmall}P)$ 
12:  $P \leftarrow P \cup \text{comp}P \cup \text{small}P \cup \text{nonAffine}P$ 
13:  $M_{csp} \leftarrow \text{generateCSP}(P)$ 
14:  $\text{valid} = \text{isEqual}(M, M_{csp})$ 

```

patterns (line 8). Basic patterns below another user-defined smaller size are also isolated (line 9) to build non-affine patterns (line 11). Basic, composite and non-affine patterns are combined to produce the output (lines 12-13), which is validated to confirm correctness (line 14). These steps are detailed below.

A. Matrix Conversion

The input matrix is converted into a container-based representation, where the entries are sorted by the sum of the row and column indices of the elements. This facilitates the

identification of starting points of patterns and their extraction. The container is designed to take a constant time on average to check the existence of a specific element [33].

B. Mining Basic Patterns

Basic patterns are mined as shown in Algorithm 2. It takes an input matrix in its container representation and produces basic patterns. The algorithm takes three parameters: T_{wf} , T_o and Th_{slim} . The first two control imperfections in patterns (Section VII-B2). The third defines “slim” patterns (Section VII-B3). The algorithm selects a *start point* with the smallest sum of row and column indices (line 1) and attempts to *expand* a block, a triangle, and a band patterns from this start point (lines 2-4) (Section VII-B1). It selects the one with the highest *score*, defined as the number of non-zero elements in the pattern less the number of imperfections weighted by a penalty factor $c \in [1, 2]$ (line 5). The score function favors patterns with more non-zero elements, with user-specified parameter c that favors patterns with fewer imperfections among patterns with similar sizes.

The mining of basic patterns is repeated by Algorithm 1 (lines 2-5) until no more non-zero elements exist outside patterns.

Algorithm 2 Basic Pattern Mining

In: M_c , **Parameters:** T_{wf}, T_o, Th_{slim}

Out: p

```

1:  $sp \leftarrow \text{getStartPoint}(M_c)$ 
2:  $b \leftarrow \text{expandBlock}(sp, M_c)$ 
3:  $t \leftarrow \text{expandTriangle}(sp, M_c)$ 
4:  $d \leftarrow \text{expandBand}(sp, M_c)$ 
5:  $p \leftarrow \text{findBest}(b, t, d, c)$ 

```

1) *Pattern Expansion*: Pattern expansion is the process of starting with a single element as the seed of a pattern, and then repeatedly exploring the neighboring area to find non-zero elements that can be added to this seed pattern. The exploration area is referred to as the *search area*, and if it contains a sufficient number of non-zero elements, they are added to

increase the size of the pattern. The process is repeated with the newly formed pattern until no more elements can be added.

The search area depends on the pattern type. For block patterns, the area is the L-shape area to the right and bottom of the existing pattern. For triangle patterns, it is the vertical line area on the right of up triangles or the horizontal line area below down triangles. For up bands, the search area is the diagonal line on the right of the band plus the horizontal line area below the band. For down bands, it is the diagonal below the band and the vertical line to its right.

The process of pattern expansion is illustrated using the example shown in Figure 13 with a block pattern (a). The starting point is the top left corner and the elements surrounding it that can potentially be part of a block are identified (b). Since these elements are non-zero, they are added to the pattern to form a 2×2 block. The elements surrounding this block are now examined (c). They are non-zeros and thus are added to form the 3×3 block (d). The elements surrounding this block are then examined. There are no non-zero elements horizontally, and the pattern is not expanded in this direction. In contrast, there are non-zero elements vertically, and the pattern is expanded with one element imperfection, resulting in the final block pattern (e).

2) *Imperfection Tolerance*: Imperfection tolerance sets the minimum requirement to expand a pattern using two parameters: *wavefront* tolerance T_{wf} , which is the maximum percentage of zeros allowed in the search area, and *overall* tolerance T_o , which is the maximum percentage of zeros allowed in the pattern if the search area elements are added. Elements are added only if the percentages of zeros in the search area and in the overall pattern are less than T_{wf} and T_o respectively.

3) *Slim Pattern Threshold*: Slim patterns have largely disproportional lengths of sides, making the search area on the longer side larger than on the shorter side. When there are no more non-zero elements in the larger side of the search area, it is repeatedly and unnecessarily explored, increasing mining time. Thus, the mining algorithm stops exploring the search area of the longer side after the ratio of the larger side and smaller side is above the slim pattern threshold, Th_{slim} .

4) *Runtime Analysis*: The basic step in Algorithm 2 is looking up an element in the container, which is $O(1)$ on average. The number of look-ups is the sum of the number of element look-ups in the successive search areas of a pattern as it is grown. In the best case, these areas do not overlap, and the total number of look-ups for all patterns is proportional to n , the number of non-zero elements in the matrix, i.e., $O(n)$. In the worst case, the successive search areas of a pattern totally overlap, and the total number of look-ups is $O(n^2)$. However, this worst case is mitigated by the slim pattern threshold that reduces look-ups when pattern sides are disproportionate, making the average runtime complexity $O(n)$ [33].

C. Composite Pattern Mining

Composite patterns are mined using Algorithm 3. It takes all patterns with fewer non-zeros than a certain threshold as input and generates the composite patterns constructed, plus the unused basic patterns. There are two parameters: Th_{min}

and Th_{size} . The first is the minimum number of basic patterns in composite patterns. The second is the minimum number of elements, including zeros, in composite patterns.

Algorithm 3 Composite Pattern Mining

In: *smallP*, **Parameters:** Th_{min}, Th_{size}

Out: *compP*, *unusedP*

```

1:  $sg \leftarrow groupOnShape(smallP)$ 
2:  $compP \leftarrow \{\}$ 
3: for  $g \in sg$  do
4:    $compP \leftarrow compP \cup findHorizontal(g)$ 
5:    $g \leftarrow g - compP$ 
6:    $compP \leftarrow compP \cup findVertical(g)$ 
7:    $g \leftarrow g - compP$ 
8:    $compP \leftarrow compP \cup findDiagonal(g)$ 
9:    $unusedP \leftarrow g - compP$ 

```

The algorithm starts by grouping patterns with identical sizes and shapes (line 1). Within each group of patterns (line 3), the algorithm further divides the patterns based on the row indices of the starting elements. Then, the algorithm exhaustively searches for sets of patterns on the same row with equal differences in column indices between neighboring patterns (line 4). The composite pattern is generated only if the number of basic patterns is above the threshold Th_{min} and the size threshold Th_{size} is met. Basic patterns used for building composite patterns are removed (line 5) before the algorithm repeats the process for patterns on the same columns (lines 6-7) and diagonals (lines 8-9).

The runtime complexity is proportional to m , the number of candidate patterns to build composite patterns. Given a group of patterns with identical size and shape, the algorithm picks a pair of patterns as reference, and finds other patterns with the same difference in row and column indices from the reference patterns. With $O(m^2)$ pairs of patterns and $O(m)$ remaining patterns in a group, the total time complexity is $O(m^3)$.

D. Non-affine Pattern Construction

Non-affine patterns are built using Algorithm 4. It takes all patterns with fewer non-zeros than a certain threshold as input and generates non-affine patterns as output. There is one parameter: s_{na} , which defines the size of the square region each non-affine pattern covers.

Algorithm 4 Non-affine Pattern Construction

In: *finalSmallP*, **Parameters:** s_{na}

Out: *nonAffineP*

```

1:  $elems \leftarrow convertToElem(finalSmallP)$ 
2:  $nonAffineP \leftarrow initNonAffine()$ 
3: for  $e \in elems$  do
4:    $insertToNonAffine(nonAffineP, e)$ 
5:  $nonAffineP \leftarrow rmEmpty(nonAffineP)$ 

```

The algorithm converts all input patterns into a set of individual elements (line 1). It allocates an empty non-affine pattern for each $s_{na} \times s_{na}$ region in the sparse matrix (line 2) and inserts each element into the correct non-affine pattern

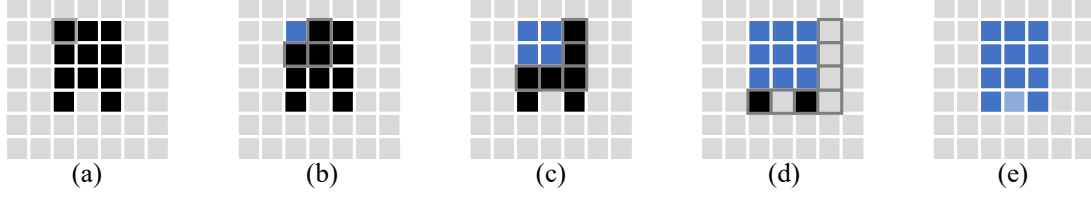


Fig. 13: Example pattern expansion

Name	Size	Density	Domain
conf5_4-8x8-05	49k x 49k	0.08%	Quantum chemistry
consph	83k x 83k	0.08%	2D/3D problem
cop20k_A	121k x 121k	0.02%	2D/3D problem
kkt_power	2m x 2m	0.0003%	Power optimization
linsp_131	131 x 131	3%	Fluid dynamic
mc2depi	525k x 525k	0.00076%	2D/3D problem
mcca	180 x 180	8%	2D/3D problem
pdb1HYS	36k x 36k	0.3%	Graph
rma10	47k x 47k	0.1%	Fluid dynamic
aft02	8k x 8k	0.2%	Acoustic
brack2	63k x 63k	0.018%	2D/3D problem
L-9	18k x 18k	0.022%	2D/3D problem
lhr07	7k x 7k	0.32%	Chemical simulation
lhr71c	70k x 70k	0.03%	Chemical simulation
mono_500Hz	169k x 169k	0.018%	Acoustic
ch8-8-b5	564k x 376k	0.002%	Combinatorial
ifiss_mat	96k x 96k	0.04%	Fluid dynamic
mark3jac140sc	64k x 64k	0.009%	Economic
pcrystk02	14k x 14k	0.5%	Duplicate material
Raj1	264k x 264k	0.002%	Circuit simulation
rajat24	358k x 358k	0.0015%	Circuit simulation
cont11_1	1.5m x 2m	0.0002%	Linear programming
HTC_336_9129	226k x 226k	0.0015%	Power network
S80PI_n	4k x 4k	0.06%	Model reduction
ts-palko	22k x 47k	0.1%	Linear programming
ww_vref_6405	13k x 13k	0.029%	Model reduction

TABLE I: Evaluation sparse matrices

based on its indices (lines 3-4). Empty non-affine patterns are removed before the output is returned (line 5).

VIII. EVALUATION

We implement pattern mining in C++, compile it with g++ -O3 (version 11.4.0) and evaluate it on an Ubuntu system with an 8 core Intel CPU (3.6 GHz i7-9700K) and 32 GB of memory.

A. Input Matrices

The set of 26 sparse matrices shown in Table I is utilized for the evaluation. The matrices are selected from the SuiteSparse Matrix Collection [17] to have a diversity in their sizes (ranging from 180×180 to $2m \times 2m$), densities (ranging from 0.0002% to 8%) and domains that include power optimization, fluid dynamics, circuit simulation etc. This diversity ensures fair evaluation of patterns and their mining.

B. Metrics and Methodology

Pattern mining is assessed with two metrics. The first is *pattern coverage*, which is defined as the ratio of the number of non-zero elements in patterns to the total number of non-zero elements in a matrix. Pattern coverage is reported for *small* patterns, which range in size from 3 to 9 non-zero elements, inclusive, *medium* patterns, which have 10 to 49 non-zero elements, inclusive, and *large* patterns, which comprise 50 or more non-zero elements. Patterns with 1 or 2 elements become

non-affine. Patterns smaller than medium are candidates for composite patterns. We categorize patterns by their sizes since larger patterns lead to more efficient data transfers. The second metric is *mining time*, which is the time to extract patterns on our evaluation platform.

Further, we evaluate the impact of CSP on data transfer overhead by calculating the total number of high-speed data transfer operations, the total size of data transfers, and the transfer cycle count.

The mining algorithm is evaluated with $T_{wf} = 50\%$, $T_o = 30\%$ for patterns of size $[50, 99]$, 20% for $[100, 499]$, and 10% for $[500, \infty)$. Further, $Th_{slim} = 50$, $Th_{min} = 3$, and $Th_{size} = 10$. The size of a non-affine pattern block is 32×32 . These represent typical parameter values; we explore their impact in Sections VIII-F and VIII-H.

The evaluation aims to answer the following questions:

- To what extent do affine patterns cover non-zero elements?
- How fast is pattern mining?
- What is the impact of composite patterns?
- How does pattern imperfection impact pattern coverage?
- How is data transfer cost impacted by the use of CSP compared to CSR and CSC?

C. Pattern Coverage

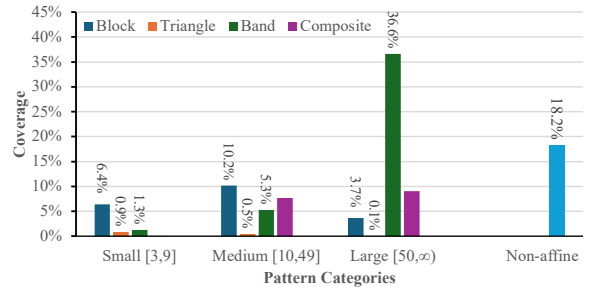


Fig. 14: Pattern coverage by pattern type

Figure 14 shows the coverage of patterns averaged over all 26 matrices in Table I, broken by pattern type and pattern size. Several observations can be made. First, the majority of pattern coverage is in affine patterns; 81.8% for affine versus 18.2% for non-affine. Thus, affine patterns are common in sparse matrices.

Second, affine patterns exist in different types, which demonstrates that our choice of pattern types is appropriate. Bands have the majority of coverage and triangles have the least. The relatively small coverage of triangles is not surprising since they are common only on diagonals of a fraction of the input matrices. Nonetheless, in matrices where they

Third, patterns appear in a range of sizes, with large patterns representing 40% of the coverage with an average size of 293 elements. Medium patterns represent 16% of the coverage with an average size of 22 elements. The large portion of medium and large patterns leads to more efficient data transfers. Finally, composite patterns make a measurable contribution to the coverage of affine patterns, almost 17%.

Matrix	Basic (sec)	Composite (sec)	Total (sec)
conf5_4-8x-05	1.1	0.0	1.1
comp20k_A	1.6	0.0	1.6
comp20k_B	1.7	0.0	1.7
kkt_power	12.6	0.0	12.6
hsp_131	0.0	0.0	0.0
mc2dpl	1.4	0.0	1.4
neuca	0.0	0.0	0.0
pdb1HYS	1.0	0.0	1.0
ma10	1.2	0.0	1.2
att02	0.1	0.0	0.1
brack2	0.7	0.0	0.7
L-9	0.0	0.0	0.0
lhr71c	0.2	0.0	0.2
mono_500Hz	2.1	0.0	2.1
ch8-8-65	10.0	0.0	10.0
htss_mat	3.1	0.0	3.1
mark3jac140c	0.5	0.0	0.5
pcrysik02	0.2	0.0	0.2
Rai1	2.8	0.0	2.8
ruja24	2.3	0.0	2.3
cont1_1	2.3	0.0	2.3
ts-palco	3.1	0.0	3.1
S80P_n	0.0	0.0	0.0
HTC_336_9129	0.2	0.0	0.2
ts_wvel_6405	0.1	0.0	0.1

Figure 15 shows the mining time for each test matrix. By and large, the mining time is small, amounting to a few seconds. However, there are some exceptions: `kkt_power`, `mono_500Hz`, `ch8-8-b5`, and notably `rajat24`. These matrices take between 10 seconds and a minute. These matrices have a large number of non-zero elements; up to 12m. This increases mining time, which is linearly proportional to the number of non-zeros. Further, `rajat24`, has $\sim 700k$ patterns, most of which are candidates for composite patterns. This leads to a high runtime, which is cubic in the number of candidates.

Pattern Categories	With Composite (%)	Without Composite (%)
Small [3,9]	8.7%	19.9%
Medium [10,49]	23.7%	16.0%
Large [50,∞)	49.4%	40.3%
Non-affine	18.2%	23.8%

Figure 16 shows the average coverage of small, medium and large affine patterns, and non-affine patterns, with and without mining composite patterns. Composite patterns clearly reduce the coverage of small and non-affine patterns in favor of medium and large ones, increasing their coverage by 8.4% on average. This demonstrates the value of mining composite patterns, and justifies the expense of increased mining time for some input matrices.

Pattern imperfections allow for larger patterns to be mined. However, they introduce overhead because of the transfer of zero elements in a patterns and computing with these zero elements. Thus, it is important to assess the impact of imperfections.

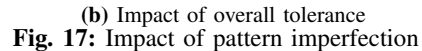


Figure 17a shows that higher wavefront tolerance T_{wf} generally results in lower non-affine coverage and improved medium and large coverage. However, high T_{wf} can reduce large pattern coverage because some large patterns lose non-zeros to medium ones, breaking the larger pattern. Figure 17b shows that higher T_o leads to a drop in medium size patterns in favor of higher large pattern coverage. With the values of T_{wf} and T_o used, the average degree of imperfection, i.e., the percentage of zero elements in patterns, is 7.1%.

This small increase in space and computation due to zero elements is offset by the significant reduction in space and data transfer with CSP (Sections VIII-G and VIII-I). It is important to note that the extent imperfections allowed is a parameter and imperfections can be disabled if desired.

Figure 18 shows for spMV the total number of data transfers, data transfer size, and data transfer cycles for CSP, compared to CSC, CSR, and BCSR with block sizes of 4×4 and 8×8 , normalized with respect to CSC. For CSC/CSR, contiguous elements in the same row/column are treated as one contiguous segment of data, incurring the same data transfer overhead as a block of the same size in CSP. For BCSR, each block is a pattern.

Figure 18a indicates that CSP incurs fewer data transfer operations compared to CSC, CSR and BCSR across all evaluation matrices. Specifically, CSP incurs on average $\sim 90\%$ fewer transfers than both CSC and CSR, and 60% to 80% fewer transfers than BCSR. Figure 18b reflects that on average,

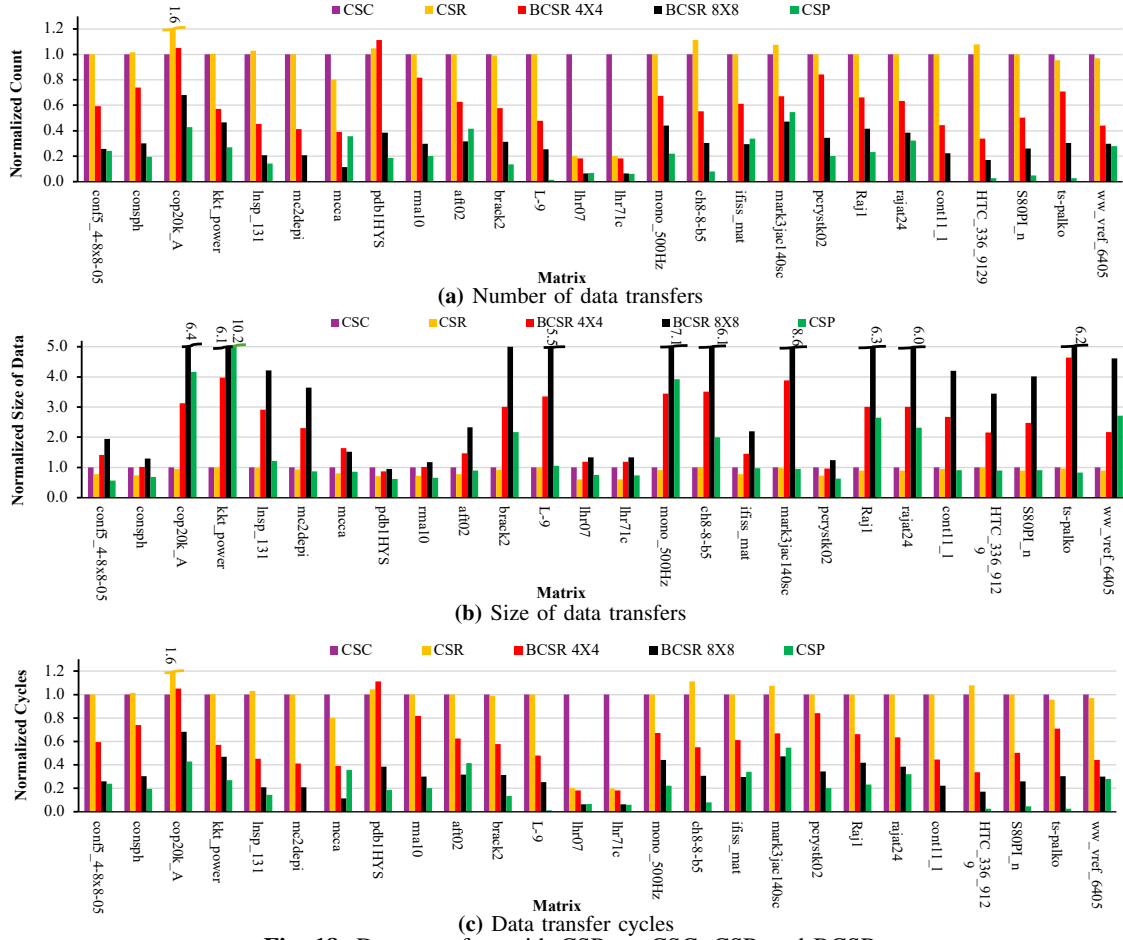


Fig. 18: Data transfers with CSP vs. CSC, CSR and BCSR

CSP requires 43% and 22% larger data transfer sizes compared to respectively CSR and CSC. In contrast, BCSR incurs 74% to 171% larger transfer size than CSP. This is mainly due to the many zero elements in BCSR blocks. Since BCSR is aimed at cache/register locality on CPUs with no control over the number of zeros included in blocks, we believe that BCSR is not as fair to compare to as CSC/CSR.

CSP incurs a similar transfer size as CSC and CSR for more than half of the matrices. Nonetheless, for eight matrices, the transfer is at least twice as much. These matrices have a large number of non-affine patterns, each of which causes one 32-element segment of x and two 32-element segments of y (one read and one write) to be transferred, including unused elements, thus increasing transfer size.

High-speed data transfers often involve a sequence of register writes to a memory-mapped I/O (MMIO) device to set the source address, the destination address, the transfer size, and trigger the transfer. On our evaluation system, each MMIO write takes 450 cycles. Thus, we estimate 1800 cycles to initiate each transfer. The speed of the transfer is typically the speed of the system bus, which is 64 bits wide. Thus, we use a transfer speed of 0.5 cycles per float. These values allow us to determine the number of cycles needed for data transfer

when using CSP, CSC, and CSR representations, shown in Figure 18c. CSP reduces data transfer cycles by 9X compared to CSR and CSC, and by 2.5X to 5X compared to BCSR, averaged over all matrices.

H. Impact of Non-Affine Patterns Size

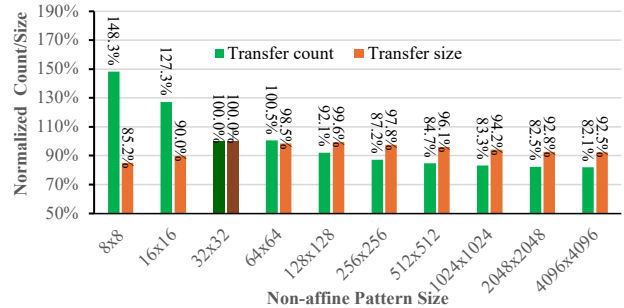


Fig. 19: Data transfer overhead for non-affine patterns

Figure 19 gives the average number of data transfers and data transfer size for non-affine pattern block sizes from 8×8 to 4096×4096 , normalized with respect to the 32×32 size. The figure shows that larger block sizes lead to fewer data transfers

due to the reduction in the number of non-affine patterns. The figure also shows that the data transfer size benefits less from increasing block sizes, but the transfer size decreases consistently. This is because the number of non-affine patterns decreases at a higher rate than the increase of data size caused by larger pattern size. Thus, the largest possible size that the SPM can accommodate should be used.

I. CSP Space Overhead

The space required by a CSP representation is a function of the number of patterns (size of index array), the type of patterns (size of parameters array), and the degree of imperfections, which introduces zeros into the values array. Averaged over all input matrices, CSP requires 30% less space compared to CSC/CSR, making CSP space-efficient.

IX. RELATED WORK

Improving the performance of sparse matrix computations is an active research area with approaches that are hardware supported [34]–[40], software oriented [13]–[16], or that combine both [41], [42]. Our work relates the most to work aiming to reduce memory accesses.

Belgin et al. [16] searches for the same-sized blocks that have the same “pattern” of non-zero and zero elements in order to reduce index array accesses. They extend earlier work on sparse matrix blocking that promotes register reuse and cache locality [13]–[15]. The above work is limited to a single pattern, e.g., only blocks, or only diagonals. and requires a priori knowledge of patterns. Our work extracts patterns (allowing some imperfections) that are not limited to equal-sized blocks and include triangles and bands. Further, our work introduces a compressed representation that improves data transfer efficiency while eliminating the use of index arrays, albeit with some hardware support.

More related are approaches to extract regularity from sparse matrices. Rodriguez et al. [43] propose the trace reconstruction engine (TRE) that takes a memory access trace and generates affine loop nests that can replicate the access trace. TRE observes the access order of the input, resulting in loop nests that are less than optimal. Further, under random access sequences, the loops become too deep and incur overhead. In contrast, our work does not need access traces and the deepest loop representing a supported pattern has a depth of 3.

Augustine et al. [44] propose codelet mining to extract rectangular blocks of non-zero elements. The blocks are grouped based on their sizes to build macro-codelets. Their work extracts only one type of pattern. In contrast, we extract multiple pattern types, allowing imperfections.

Rodriguez et al. [45] propose UZP, a sparse matrix representation based on polyhedra. They take a list of target polyhedra shapes as input and extracts matching ones in a matrix. Generic polyhedra do not guarantee efficient data transfer on accelerators without complex extensions to hardware. Further, UZP does not allow imperfections.

Efficient compressed representations exist for matrices that have their non-zero elements in a single pattern, such as block diagonal, diagonal, tri-diagonal, etc [11], [12]. These representations are limited to single patterns while we represent different patterns within a matrix.

Finally, there is work on the design of accelerators for sparse matrix computations and associated sparse matrix representations. Parashar et al. [46] introduce a sparse CNN accelerator that accommodates compressed weight matrices, where compression records the number of zeros between neighboring non-zero elements. Gondimalla et al. [47] focus on the inner product in sparse vector-vector dot product, proposing dedicated hardware for it. Kjolstad et al. [48] generate efficient kernels on CPUs for sparse computations by introducing format descriptors of tensors to denote dense and sparse dimensions, building iteration graphs, and using merge lattices to minimize zero-operand computation. Benatia et al. [49] aim to select the best out of several sparse representations using a pairwise cost-sensitive classification approach, targeting GPUs. This body of work is orthogonal to ours, where we aim not to design an accelerator but focus on data transfer efficiency to/from existing ones.

X. CONCLUDING REMARKS

We exploit sparsity patterns that exist in sparse matrices to regularize data accesses and improve data transfer efficiency to/from accelerators. Specifically, we propose an algorithm for mining patterns from sparse matrices and representing them with parameters of affine loops. Further, we propose CSP, a novel compressed representation that is based on these patterns. We implement the mining algorithm and use it to extract patterns in 26 real-world sparse matrices. Evaluation shows that patterns are indeed common and they exist in a variety of types and sizes, both basic and composite. The patterns cover the majority of non-zero elements in the matrices. Further, the patterns can be efficiently extracted. Finally, the use of CSP with an extended AGU design leads to up to 90% reduction in data transfer overhead, compared to CSR and CSC, for the classical spMV computation. This validates our approach of exploiting patterns to improve data transfer efficiency.

Several directions for future work exist. It is possible to mine other types of patterns, e.g., bands and triangles with different orientations or trapezoids, and other forms of composite patterns. It is also possible to parallelize the mining algorithm for better performance. Further, it is possible to improve the processing of non-affine patterns to reduce their overhead. Finally, it is interesting to extend the approach to dynamically generated sparse matrices.

REFERENCES

- [1] M. J. Abraham, T. Murtola, R. Schulz, S. Páll, J. C. Smith, B. Hess, and E. Lindahl, “Gromacs: High performance molecular simulations through multi-level parallelism from laptops to supercomputers,” *SoftwareX*, vol. 1-2, pp. 19–25, 2015.
- [2] P. Sadayappan and V. Visvanathan, “Efficient sparse matrix factorization for circuit simulation on vector supercomputers,” *Trans. on Computer-Aided Design of Integrated Circuits and Systems*, vol. 8, no. 12, pp. 1276–1285, 1989.
- [3] E. H. Rubensson, E. Rudberg, and P. Salek, “Sparse matrix algebra for quantum modeling of large systems,” in *Applied Parallel Computing*, B. Kågström, E. Elmroth, J. Dongarra, and J. Waśniewski, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2007, pp. 90–99.
- [4] A. George, “Sparse matrix aspects of the finite element method,” in *Computing Methods in Applied Sciences and Engineering*, R. Glowinski and J. L. Lions, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 1976, pp. 3–22.
- [5] E. Frantar and D. Alistarh, “Sparsegpt: Massive language models can be accurately pruned in one-shot,” 2023. [Online]. Available: <https://arxiv.org/abs/2301.00774>

- [6] Y. Li, Y. Yu, Q. Zhang, C. Liang, P. He, W. Chen, and T. Zhao, "Losparse: Structured compression of large language models based on low-rank and sparse approximation," 2023. [Online]. Available: <https://arxiv.org/abs/2306.11222>
- [7] A. Agarwalla, A. Gupta, A. Marques, S. Pandit, M. Goin, E. Kurtic, K. Leong, T. Nguyen, M. Salem, D. Alistarh, S. Lie, and M. Kurtz, "Enabling high-sparsity foundational llama models with efficient pretraining and deployment," 2024. [Online]. Available: <https://arxiv.org/abs/2405.03594>
- [8] Y. Saad, *Iterative Methods for Sparse Linear Systems*, 2nd ed. Society for Industrial and Applied Mathematics, 2003.
- [9] K. E. Gustafson and R. A. Brooks, "An overview of matrix techniques in the ellpack problem-solving environment," *Mathematics of Computation*, vol. 34, no. 150, pp. 469–476, 1980.
- [10] M. Kreutzer, G. Hager, G. Wellein, H. Fehske, and A. R. Bishop, "A unified sparse matrix data format for efficient general sparse matrix-vector multiplication on modern processors with wide simd units," *SIAM Journal on Scientific Computing*, vol. 36, no. 5, pp. 401–423, 2014.
- [11] R. Barrett, M. Berry, T. F. Chan, J. Demmel, J. Donato, J. Dongarra, V. Eijkhout, R. Pozo, C. Romine, and H. V. der Vorst, *Templates for the Solution of Linear Systems: Building Blocks for Iterative Methods*. SIAM, 1994.
- [12] J. Choi, A. Singh, and R. Vuduc, "Model-driven autotuning of sparse matrix-vector multiply on gpus," *Sigplan Notices - SIGPLAN*, vol. 45, pp. 115–126, 05 2010.
- [13] E.-J. Im and K. Yelick, "Optimizing sparse matrix computations for register reuse in sparsity," in *Proc. of Int'l Conf. on Computational Science*, V. N. Alexandrov, J. J. Dongarra, B. A. Juliano, R. S. Renner, and C. J. K. Tan, Eds., 2001, pp. 127–136.
- [14] R. Vuduc, J. W. Demmel, and K. A. Yelick, "Oski: A library of automatically tuned sparse matrix kernels," *Journal of Physics: Conference Series*, vol. 16, no. 1, p. 521, jan 2005.
- [15] R. W. Vuduc and H.-J. Moon, "Fast sparse matrix-vector multiplication by exploiting variable block structure," in *High Performance Computing and Communications*, L. T. Yang, O. F. Rana, B. Di Martino, and J. Dongarra, Eds., 2005, pp. 807–816.
- [16] M. Belgin, G. Back, and C. J. Ribbens, "Pattern-based sparse matrix representation for memory-efficient smvm kernels," in *Proc. of the Int'l Conf. on Supercomputing*, 2009, pp. 100–109.
- [17] T. Davis, Y. Hu, and S. Kolodziej, "Suitesparse matrix collection," 2025, accessed: 2025-04-19. [Online]. Available: <https://sparse.tamu.edu/>
- [18] F. Schukki, M. Schaffner, and L. Benini, "NTX: An energy-efficient streaming accelerator for floating-point generalized reduction workloads in 22 nm fd-soi," in *Design, Automation & Test in Europe Conference & Exhibition*, 2019, pp. 662–667.
- [19] J. Lee and J. Lee, "NP-CGRA: Extending cgras for efficient processing of light-weight deep neural networks," in *Design, Automation & Test in Europe Conference & Exhibition*, 2021, pp. 1408–1413.
- [20] P. Kasgen, M. Messelka, and M. Weinhardt, "HiPReP: High-performance reconfigurable processor - architecture and compiler," in *Int'l Conf. on Field-Programmable Logic and Applications*, 2021, pp. 380–381.
- [21] Y. Tang and C. Wang, "Performance modeling on DaVinci AI Core," *J. of Parallel and Distributed Computing*, vol. 175, pp. 134–149, 2023.
- [22] Y.-H. Chen, T.-J. Yang, J. Emer, and V. Sze, "Eyeriss v2: A flexible accelerator for emerging deep neural networks on mobile devices," 2019. [Online]. Available: <https://arxiv.org/abs/1807.07928>
- [23] M. D. Ruiz Noguera and C. McCabe, "Tutorial: Introduction to the AMD Versal ACAP adaptable intelligent engine and to its programming model," in *Proc. of the Int'l Conf. on Hardware/Software Codesign and System Synthesis*, 2024, p. 7.
- [24] A. Reuther, P. Michaleas, M. Jones, V. Gadepally, S. Samsi, and J. Kepner, "Survey of machine learning accelerators," in *Proc. of the High Performance Extreme Computing Conf.*, 2020, pp. 1–12.
- [25] Z. Xu, "DaVinci: A scalable architecture for neural network computing," 2020, accessed: 2025-09-13. [Online]. Available: <https://www.cmc.ca/wp-content/uploads/2020/03/Zhan-Xu-Huawei.pdf>
- [26] F. Gao, X. Yue, C. Tang, H. Chen, K.-T. A. Wang, and T. S. Abdelrahman, "On-the-fly data layout conversion for GEMM on AI accelerators," in *Proc. of the IEEE Int'l Symp. on Parallel and Distributed Processing with Applications*, 2024, pp. 484–493.
- [27] W. Xue, B. Xu, K. Yang, Y. Liu, D. Fan, P. Xu, and Y. Tian, "SGEMM-cube: Emulating FP32 GEMM on Ascend NPU's using FP16 cube units with precision recovery," 2025. [Online]. Available: <https://arxiv.org/abs/2507.23387>
- [28] AMD, "Versal adaptive SoC AI engine architecture manual," 2023, accessed: 2025-09-13. [Online]. Available: <https://docs.amd.com/r/en-US/am009-versal-ai-engine/Functional-Overview>
- [29] M. A. Shami and A. Hemani, "Address generation scheme for a coarse grain reconfigurable architecture," in *Proc. of the Int'l Conf. on Application-specific Systems, Architectures and Processors*, 2011, pp. 17–24.
- [30] N. Farahani, A. Hemani, H. Sohofi, S. M. Jafri, M. A. Tajammul, and K. Paul, "Parallel distributed scalable runtime address generation scheme for a coarse grain reconfigurable computation and storage fabric," *Microprocessors and Microsystems*, vol. 38, no. 8, Part A, pp. 788–802, 2014.
- [31] N. Sato and W. F. Tinney, "Techniques for exploiting the sparsity or the network admittance matrix," *IEEE Trans. on Power Apparatus and Systems*, vol. 82, no. 69, pp. 944–950, 1963.
- [32] Microsoft, "Index-accera," Available at <https://microsoft.github.io/Accera/Manual/>, 2023, accessed 2024-12-31.
- [33] Y. Su, "Exploiting sparsity patterns in sparse matrices to improve data transfer efficiency on accelerators," Master's thesis, University of Toronto, 2025.
- [34] S. Pal, J. Beaumont, D.-H. Park, A. Amarnath, S. Feng, C. Chakrabarti, H.-S. Kim, D. Blaauw, T. Mudge, and R. Dreslinski, "Outerspace: An outer product based sparse matrix multiplication accelerator," in *Proc. of the Int'l Symp. on High Performance Computer Architecture*, 2018, pp. 724–736.
- [35] K. Hegde, H. Asghari-Moghaddam, M. Pellauer, N. Crago, A. Jaleel, E. Solomonik, J. Emer, and C. W. Fletcher, "ExTensor: An accelerator for sparse tensor algebra," in *Proc. of the Int'l Symp. on Microarchitecture*, 2019, pp. 319–333.
- [36] Z. Zhang, H. Wang, S. Han, and W. J. Dally, "Sparch: Efficient architecture for sparse matrix multiplication," in *Proc. of the Int'l Symp. on High Performance Computer Architecture*, 2020, pp. 261–274.
- [37] Y. N. Wu, P.-A. Tsai, S. Muralidharan, A. Parashar, V. Sze, and J. Emer, "Highlight: Efficient and flexible dnn acceleration with hierarchical structured sparsity," in *Proc. of the Int'l Symp. on Microarchitecture*, 2023, pp. 1106–1120.
- [38] N. Srivastava, H. Jin, J. Liu, D. Albonese, and Z. Zhang, "Matraport: A sparse-sparse matrix multiplication accelerator based on row-wise product," in *Proc. of the Int'l Symp. on Microarchitecture*, 2020, pp. 766–780.
- [39] L. Song, Y. Chi, A. Sohrabzadeh, Y.-k. Choi, J. Lau, and J. Cong, "Sextans: A streaming accelerator for general-purpose sparse-matrix dense-matrix multiplication," in *Proc. of the Int'l Symp. on Field-Programmable Gate Arrays*, 2022, pp. 65–77.
- [40] X. Xie, Z. Liang, P. Gu, A. Basak, L. Deng, L. Liang, X. Hu, and Y. Xie, "Spacea: Sparse matrix vector multiplication on processing-in-memory accelerator," in *Proc. of the Int'l Symp. on High-Performance Computer Architecture*, 2021, pp. 570–583.
- [41] O. Zachariadis, N. Satpute, J. Gómez-Luna, and J. Olivares, "Accelerating sparse matrix-matrix multiplication with GPU tensor cores," *Computers & Electrical Engineering*, vol. 88, p. 106848, 2020.
- [42] F. Gremse, A. Höfter, L. O. Schwen, F. Kiessling, and U. Naumann, "Gpu-accelerated sparse matrix-matrix multiplication by iterative row merging," *SIAM Journal on Scientific Computing*, vol. 37, no. 1, pp. C54–C71, 2015.
- [43] G. Rodríguez, J. M. Andión, M. T. Kandemir, and J. Touriño, "Trace-based affine reconstruction of codes," in *Proc. of the Int'l Symp. on Code Generation and Optimization*, 2016, pp. 139–149.
- [44] T. Augustine, J. Sarma, L.-N. Pouchet, and G. Rodríguez, "Generating piecewise-regular code from irregular structures," in *Proc. of the Conf. on Programming Language Design and Implementation*, 2019, pp. 625–639.
- [45] A. Rodríguez-Iglesias, S. T. Tongli, E. Tucker, L.-N. Pouchet, G. Rodríguez, and J. Touriño, "Modular construction and optimization of the UZP sparse format for SpMV on CPUs," *Proc. ACM Program. Lang.*, vol. 9, no. PLDI, 2025.
- [46] A. Parashar, M. Rhu, A. Mukkara, A. Puglielli, R. Venkatesan, B. Khailany, J. Emer, S. W. Keckler, and W. J. Dally, "SCNN: An accelerator for compressed-sparse convolutional neural networks," in *Proc. of the Int'l Symp. on Computer Architecture*, 2017, pp. 27–40.
- [47] A. gondimalla, N. Chesnut, M. Thottethodi, and T. N. Vijaykumar, "SparTen: A sparse tensor accelerator for convolutional neural networks," in *Proc. of the Int'l Symp. on Microarchitecture*, 2019, pp. 151–165.
- [48] F. Kjolstad, S. Kamil, S. Chou, D. Lugato, and S. Amarasinghe, "The tensor algebra compiler," *Proc. ACM Program. Lang.*, vol. 1, Oct. 2017.
- [49] A. Benatia, W. Ji, Y. Wang, and F. Shi, "BestSF: A sparse meta-format for optimizing SpMV on GPU," *ACM Trans. Archit. Code Optim.*, vol. 15, no. 3, Sep. 2018.